ERDMS Project Guide

For the Remaining Deliverables

This guide is for the team members who will be creating the remaining diagrams and documents. Read each section fully before starting — understanding *why* you're making something is just as important as *how* to make it.

---

1. Context Diagram

What Is It?

A Context Diagram shows your **entire system as a single box** and how it interacts with the outside world. It does not show any internal details — just what goes in and what comes out.

Think of it as a **black box view.** You don't care what happens inside the system yet, only who talks to it and what they send or receive.

Key Components

- **One box** in the center — that's your entire ERDMS system
- **External entities** around it — the people or things that interact with it
- **Arrows** showing data flowing in and out

Your External Entities

Based on the CRD, you have two main actors:

Entity	What They Send In	What They Get Back
Admin	Login credentials, uploaded files, user management actions, folder actions	Dashboard data, audit logs, confirmations
User	Login credentials, download requests, search queries	Documents, search results, profile info

Common Mistakes to Avoid

- **Don't show internal processes** — no database, no backend, no modules inside the box
- **Don't connect external entities to each other** — all arrows must go through the system
- **Don't forget labels on arrows** — every flow needs a name

Recommended Tool

draw.io (diagrams.net) — free, no account needed. Use the basic shapes: rectangle for the system, rectangle or oval for external entities, arrows for data flows.

---

2. Use Case Diagram

What Is It?

A Use Case Diagram shows **who uses the system and what they can do.** It maps actors to actions — nothing more.

Think of it as a **visual list of features** organized by who has access to them.

Key Components

- **Actors** — the people who use the system (stick figures on the outside)
- **Use Cases** — the actions they can perform (ovals inside the system boundary)
- **System Boundary** — a rectangle that contains all the use cases
- **Lines** — connecting actors to the use cases they can perform

Your Actors

You have two actors directly from the CRD:

- **Admin**
- **User**

Your Use Cases

Pull these directly from CRD Section VI. Each bullet point under functional requirements is essentially a use case:

Admin Use Cases

- Login / Logout
- Create User Account
- Update User Details
- Delete User Account
- Activate / Deactivate User
- Create Folder

- Rename Folder
- Delete Folder
- Upload Document
- Edit Document Metadata
- Delete Document
- View Dashboard
- View Audit Logs

User Use Cases

- Login / Logout
- View Profile
- Update Personal Information
- Change Password
- View Documents
- Download Document
- Search Documents
- Filter Documents by Category

Important Relationships to Know

Include — when one use case always requires another to happen first. Draw a dashed arrow labeled <<include>>.

Example: "Download Document" always requires "View Document" first.

Download Document —<<include>>—→ View Document

Extend — when one use case optionally adds behavior to another. Draw a dashed arrow labeled <<extend>>.

Example: "Search Documents" can optionally be extended by "Filter by Category."

Filter by Category —<<extend>>—→ Search Documents

For a school project, include and extend are optional but using them shows deeper understanding.

Common Mistakes to Avoid

- **Don't draw arrows between use cases unless using include/extend**
- **Don't forget the system boundary rectangle**
- **Keep use case names as verb phrases** — "Upload Document" not just "Document"

Recommended Tool

draw.io again works well. Use the UML shape library for proper stick figures and ovals.

---

3. SQL Schema

What Is It?

The SQL Schema is the actual **code that creates your database tables.** It translates your ERD directly into MySQL statements.

Think of it as your ERD **coming to life in code.**

How to Write It

Each table from your ERD becomes a CREATE TABLE statement. The columns become the fields, and the relationships become FOREIGN KEY constraints.

The Order Matters

You must create tables in the right order — **referenced tables first, then the tables that reference them.** This is because a foreign key can't point to a table that doesn't exist yet.

Correct order for your project:

1. Roles (nothing depends on this first)
2. Users (depends on Roles)
3. Folders (depends on Users)
4. Documents (depends on Folders and Users)
5. Logs (depends on Users and Documents)

What Each Part Means

When writing a column, you define three things:

- **Name** — what it's called

- **Data type** — what kind of data it holds
- **Constraints** — rules it must follow

Common data types you'll use:

Type	Use It For
INT	Numbers, IDs
VARCHAR(n)	Short text, names, emails
TEXT	Long text, descriptions
ENUM	Fixed options like status
TIMESTAMP	Date and time

Common constraints:

Constraint	What It Does
PRIMARY KEY	Marks the unique identifier
NOT NULL	Field cannot be empty
UNIQUE	No two rows can have the same value
AUTO_INCREMENT	ID number increases automatically
FOREIGN KEY	Links to another table
DEFAULT	Sets a default value if none is given

Reference Schema Based on Your ERD

---

4. Project Plan

What Is It?

A Project Plan is a **timeline of your entire development process.** It shows what tasks need to be done, in what order, and who is responsible.

Think of it as your team's **roadmap from start to finish.**

Development Phases based on your CRD's Waterfall methodology:

Phase	What Happens Here
1. Requirements Analysis	Review CRD, finalize scope, clarify with instructor

2. System Design	Finish all diagrams, ERD, SQL schema
3. Frontend Development	Build Angular UI screens
4. Backend Development	Build Node.js + Express API
5. Database Implementation	Set up MySQL, run SQL schema
6. Integration	Connect frontend to backend to database
7. Testing	Test all features, fix bugs
8. Documentation	Write final documentation
9. Deployment / Presentation	Deploy locally, present to instructor

How to Present It

The most common formats are:

Simple Table — list tasks, assigned member, and target date in a table. Easiest to make.

Gantt Chart — a bar chart where each task is a horizontal bar spanning its duration. More visual and professional.

For a school project a simple table is acceptable, but a Gantt chart will look more impressive.

Tips for a Good Project Plan

- Be **realistic** with your dates — don't compress everything into one week
- Assign **specific people** to each task, not just "everyone"
- Account for **overlapping tasks** — frontend and backend can be developed simultaneously
- Include **buffer time** before the deadline for unexpected problems